

Let me give you a thorough visual explanation of the entire flow.
Let me build this as a comprehensive interactive visual guide.

1
Install Ollama

2
Docker locally

3
Build & push
ACR

4
Deploy Azure

5
Verify & test

STEP 1 — INSTALL OLLAMA ON YOUR MACHINE

What is Ollama?

Ollama is a runtime that downloads and serves open-source LLMs (Llama 3, Mistral, Gemma, etc.) on your own machine via a simple REST API on port 11434. Your Streamlit app calls `http://localhost:11434/api/chat` — no API key, no cloud cost.

PICK YOUR OS

MACOS / LINUX

```
# Install
brew install ollama
# or via curl
curl -fsSL
https://ollama.com/install.sh | sh

# Start the server
ollama serve
```

WINDOWS

Download the installer from ollama.com/download and run it. Ollama starts automatically as a system tray app.

```
# Then in PowerShell
ollama pull llama3.2
ollama serve
```

CHOOSE A MODEL (PULL ONE)

llama3.2

3B params · ~2 GB

Recommended for demo

mistral

7B params · ~4 GB

Best quality/speed

phi3

3.8B · ~2.3 GB

Lightweight

gemma2

9B params · ~5 GB

Google, very capable

llama3.1

8B params · ~5 GB

Balanced

llama3.1:70b

70B · ~48 GB RAM

Needs GPU

Pull your chosen model (downloads weights once)

```
ollama pull llama3.2
```

Verify it works

```
curl http://localhost:11434/api/tags
```

Quick chat test

```
ollama run llama3.2 "Say hello in one sentence"
```

Copy

Click any model above to update the pull command

Models are stored in `~/.ollama/models`. Once pulled, they work fully offline.

----- 2 step -----

1
Install Ollama

2
Docker locally

3
Build & push
ACR

4
Deploy Azure

5
Verify & test

STEP 2 — RUN EVERYTHING LOCALLY WITH DOCKER COMPOSE

docker compose up --build

ollama
port 11434
models volume

models

ollama-pull
one-shot helper
pulls llama3.2

API calls

app (Streamlit)
port 8501
OLLAMA_BASE_URL=

```
# 1. Copy environment config
cp .env.example .env

# 2. Set your model in .env
OLLAMA_BASE_URL=http://localhost:11434
OLLAMA_MODEL=llama3.2

# 3. Start all containers (Ollama + app)
docker compose up --build

# First run: downloads model (~2 GB), takes a few minutes
# Subsequent runs: uses cached ollama_models volume, instant start
```

Copy

Why two Ollama containers?

ollama is the long-running server. ollama-pull is a one-shot helper that waits 5 seconds, runs `ollama pull llama3.2`, then exits. This ensures the model is downloaded before the app starts sending requests.

```
# Verify all three containers are running
docker compose ps

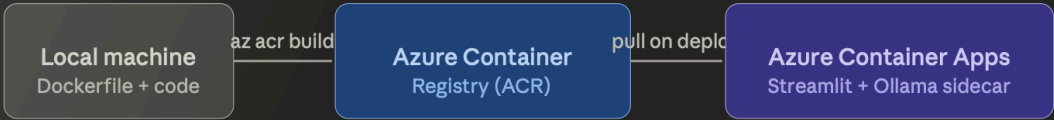
# Stream logs from the Ollama container
docker compose logs -f ollama

# Open the app
open http://localhost:8501
```

Copy

----- 3rd step -----

STEP 3 — BUILD YOUR IMAGE AND PUSH TO AZURE CONTAINER REGISTRY



```
graph LR; A[Local machine  
Dockerfile + code] -- "az acr build" --> B[Azure Container  
Registry (ACR)]; B -- "pull on deployment" --> C[Azure Container Apps  
Streamlit + Ollama sidecar]
```

- 1 Login to Azure**

```
az login
az account set --subscription "Your Subscription Name"
```

Copy
- 2 Create a Resource Group and ACR**

```
# Create resource group
az group create --name ai-voice-agent-rg --location eastus

# Create Container Registry (Basic SKU = cheapest)
az acr create --resource-group ai-voice-agent-rg --name aivoiceagentacr --sku Basic --admin-enabled true
```

Copy
- 3 Build and push your Streamlit app image**

```
# Option A: Build in Azure (no Docker needed locally)
az acr build --registry aivoiceagentacr --image ai-voice-agent:latest .

# Option B: Build locally then push
```

Copy

↓

3

Build and push your Streamlit app image

```
# Option A: Build in Azure (no Docker needed locally)
az acr build --registry aivoiceagentacr --image ai-voice-agent:latest .

# Option B: Build locally then push
docker build -t aivoiceagentacr.azurecr.io/ai-voice-agent:latest .
az acr login --name aivoiceagentacr
docker push aivoiceagentacr.azurecr.io/ai-voice-agent:latest
```

Copy

Use Option A if possible

az acr build uploads your source code to Azure and builds in the cloud — faster, no local Docker needed, and automatically pushes to ACR.

4

Verify the image is in ACR

```
az acr repository list --name aivoiceagentacr --output table

az acr repository show-tags --name aivoiceagentacr --repository ai-voice-agent
```

Copy

Ollama image comes from Docker Hub directly

You do NOT push the Ollama image to ACR. Azure Container Apps pulls ollama/ollama:latest from Docker Hub automatically when the sidecar container starts. Only your Streamlit app image needs to be in ACR.

STEP 4 — DEPLOY TO AZURE CONTAINER APPS

Azure Container Apps — single revision



Sidecar pattern — why it matters

Both containers run in the same Container App revision. They share a network namespace, so the Streamlit app reaches Ollama at `http://localhost:11434` — zero latency, no public exposure of the Ollama port. They also share an EmptyDir volume so Ollama can cache model weights.

```
# One-command deploy using the Bicep template
az deployment group create \
  --resource-group ai-voice-agent-rg \
  --template-file azure/main.bicep \
  --parameters containerImage=aivoiceagentacr.azurecr.io/ai-voice-agent:latest \
  --parameters ollamaModel=llama3.2 \
  --parameters sendgridApiKey=SG.xxxxx \
  --parameters productEmail=product@yourco.com \
  --parameters paymentEmail=payments@yourco.com
```

Copy

OR USE THE ONE-SHOT DEPLOY SCRIPT

```
# Interactive – prompts for all secrets securely
bash azure/deploy.sh
```

Copy

KEY ENVIRONMENT VARIABLES SET BY BICEP

VARIABLE	VALUE IN AZURE
OLLAMA_BASE_URL	http://localhost:11434 (sidecar shares localhost)
OLLAMA_MODEL	llama3.2 (or whichever you set)
OLLAMA_TIMEOUT	180 (generous, first request cold-starts the model)
DB_TYPE	sqlite (uses EmptyDir; swap to mysql for persistence)
APP_ENV	production
SENDGRID_API_KEY	Pulled from Key Vault secret reference

First startup is slow (~3–5 min)

When the container app starts for the first time, Ollama pulls the model weights inside the container (~2 GB for llama3.2). Subsequent restarts reuse the EmptyDir cache. For production, mount an Azure File Share as the volume instead of EmptyDir so weights survive restarts.

STEP 5 — VERIFY YOUR DEPLOYMENT AND DEBUG TIPS

● Get your app URL

Printed at end of deployment, or: `az containerapp show --name ai-voice-agent-app --resource-group ai-voice-agent-rg --query properties.configuration.ingress.fqdn -o tsv`

● Check Ollama health from Admin panel

Open the app → Admin → LLM & Config → click "Check Status". Should show ● Ollama Online with llama3.2 listed.

● Test a chat query

Go to Chat page, type "What is your return policy?" — should get a categorised Product response.

● Check container logs

`az containerapp logs show --name ai-voice-agent-app --resource-group ai-voice-agent-rg --follow`

● Trigger a test email summary

Admin panel → Email Control → "Send All Department Summaries NOW"

COMMON ISSUES AND FIXES

Ollama shows offline / timeout errors

The sidecar is still pulling the model. Wait 3–5 minutes and retry. Check logs: `az containerapp logs show ... | grep ollama`. If it keeps failing, increase `OLLAMA_TIMEOUT` to 300 in the Container App environment variables.

JSON parse errors / garbled AI responses

Small models (phi3, llama3.2) occasionally respond outside the JSON format. The app's fallback returns the raw text as the answer. Switch to `mistral` or `llama3.1` for more reliable JSON adherence. Change `OLLAMA_MODEL` in Container App env vars and restart.

Persist model weights across restarts

Replace the `EmptyDir` volume with an `Azure File Share` mounted at `/root/.ollama`. This way the model is downloaded once and survives pod restarts and scaling events. ↓

COMMON ISSUES AND FIXES

Ollama shows offline / timeout errors

The sidecar is still pulling the model. Wait 3-5 minutes and retry. Check logs: `az containerapp logs show ... | grep ollama`. If it keeps failing, increase `OLLAMA_TIMEOUT` to 300 in the Container App environment variables.

JSON parse errors / garbled AI responses

Small models (phi3, llama3.2) occasionally respond outside the JSON format. The app's fallback returns the raw text as the answer. Switch to `mistral` or `llama3.1` for more reliable JSON adherence. Change `OLLAMA_MODEL` in Container App env vars and restart.

Persist model weights across restarts

Replace the `EmptyDir` volume with an `Azure File Share` mounted at `/root/.ollama`. This way the model is downloaded once and survives pod restarts and scaling events.

Restart the container app (forces fresh pull)

```
az containerapp revision restart --name ai-voice-agent-app --resource-group ai-voice-agent-rg
```

Copy

Update `OLLAMA_MODEL` to switch models without redeploying

```
az containerapp update --name ai-voice-agent-app --resource-group ai-voice-agent-rg \
  --set-env-vars OLLAMA_MODEL=mistral
```